

SIMATS ENGINEERING

Department of Computer Science and Engineering ASSESSMENT TOOL2 - CONCEPT MAPPING

Course Code:	CSA1221	Course Title:	Computer Architecture
CO Assessed:	CO1	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	K NISHALINI	Reg.No.: 192521301	
Department:	IT	Date: 22/07/2026	

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

Code of Conduct:

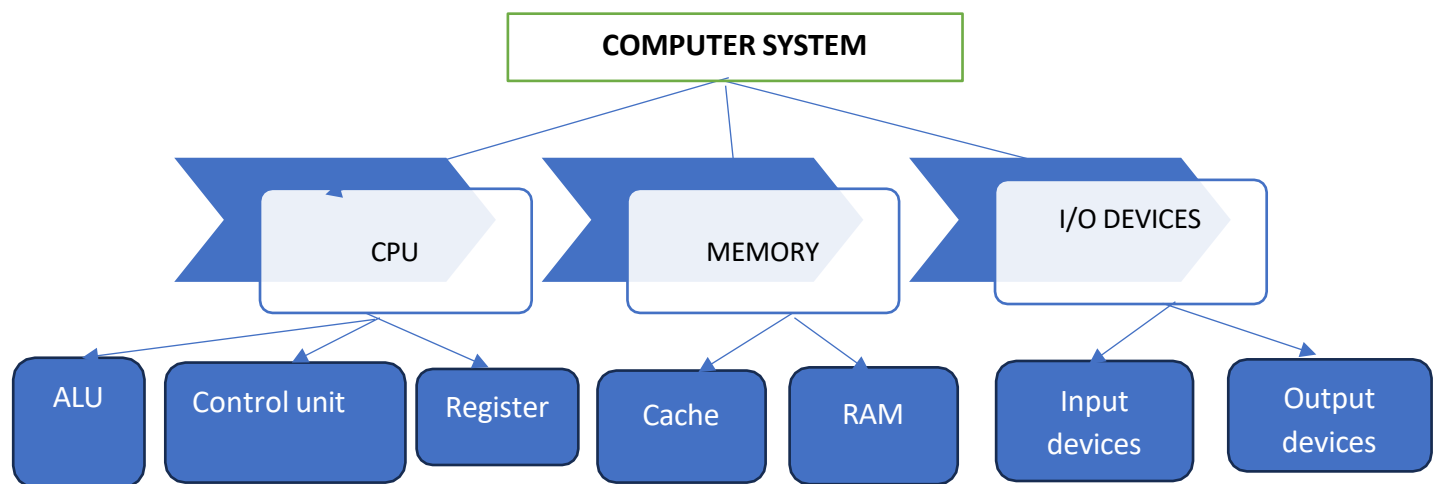
I NISHALINI K (192521301) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

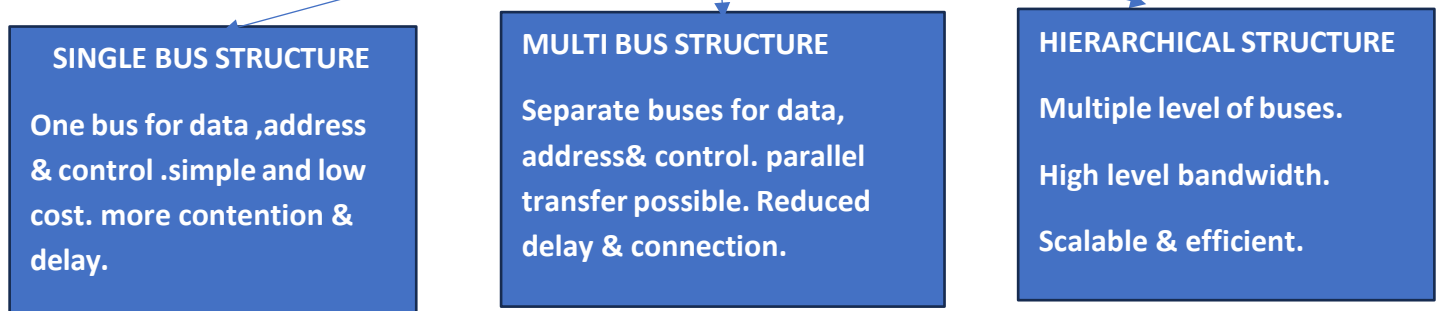
NISHALINI K

ANNEXURE A

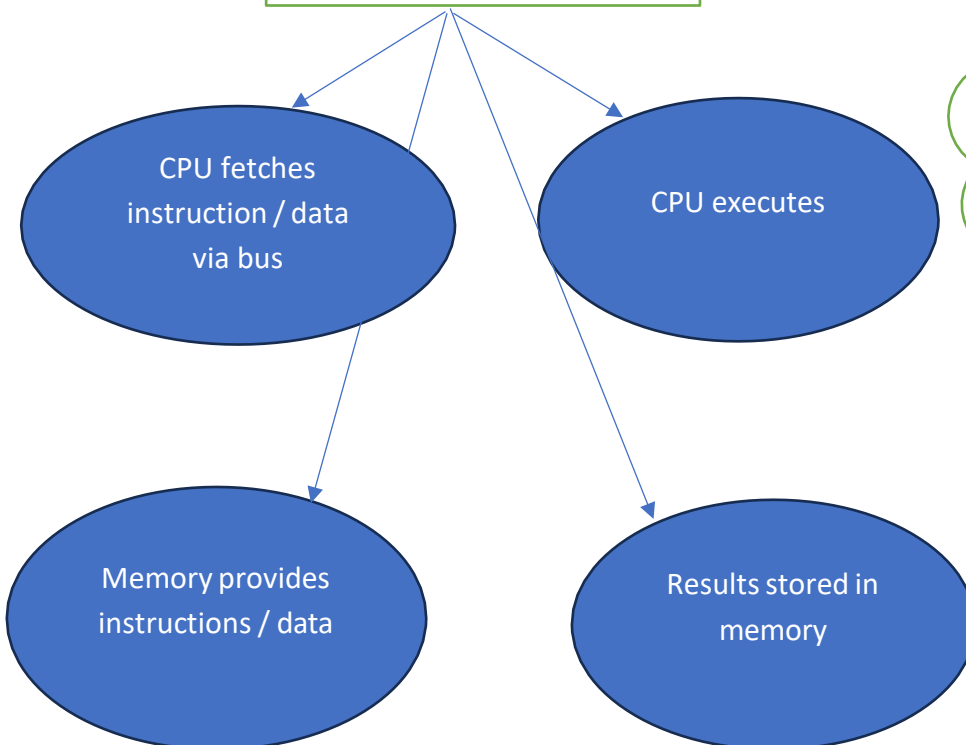
1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.



BUS STRUCTURES



FLOW DURING EXECUTION



DATA & CONTROL SIGNAL FLOW

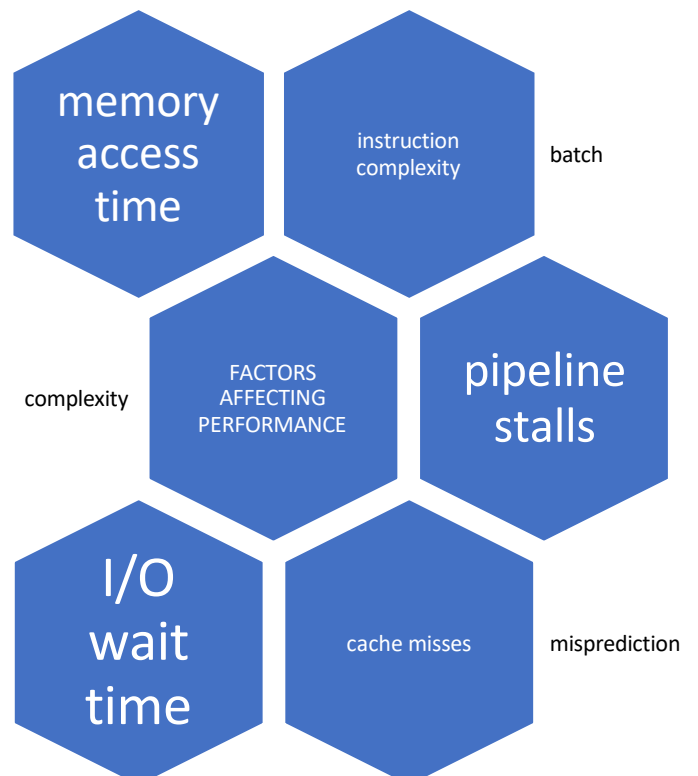
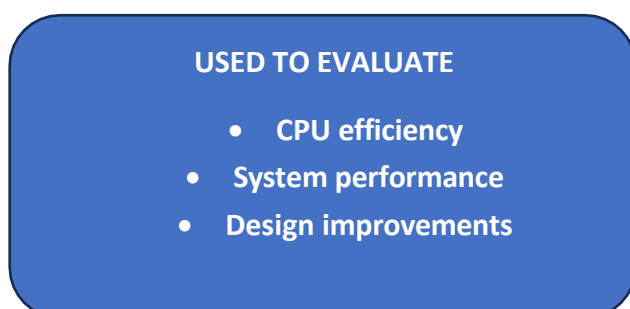
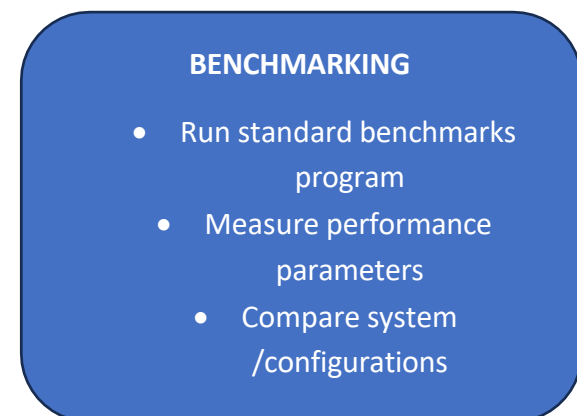
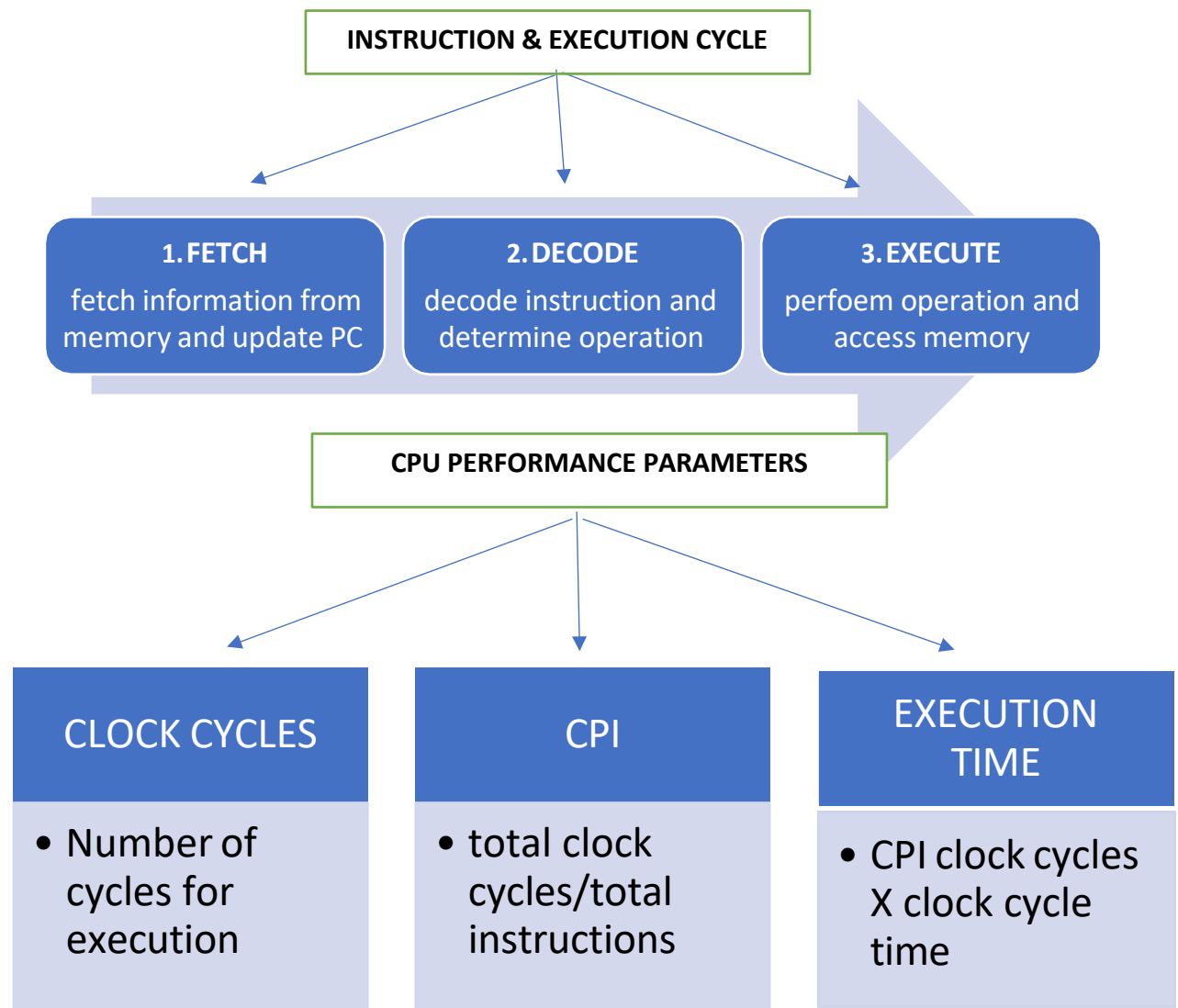
- Data bus
- Address bus
- Control bus

HOW BUS STRUCTURES IMPROVES PERFORMANCE

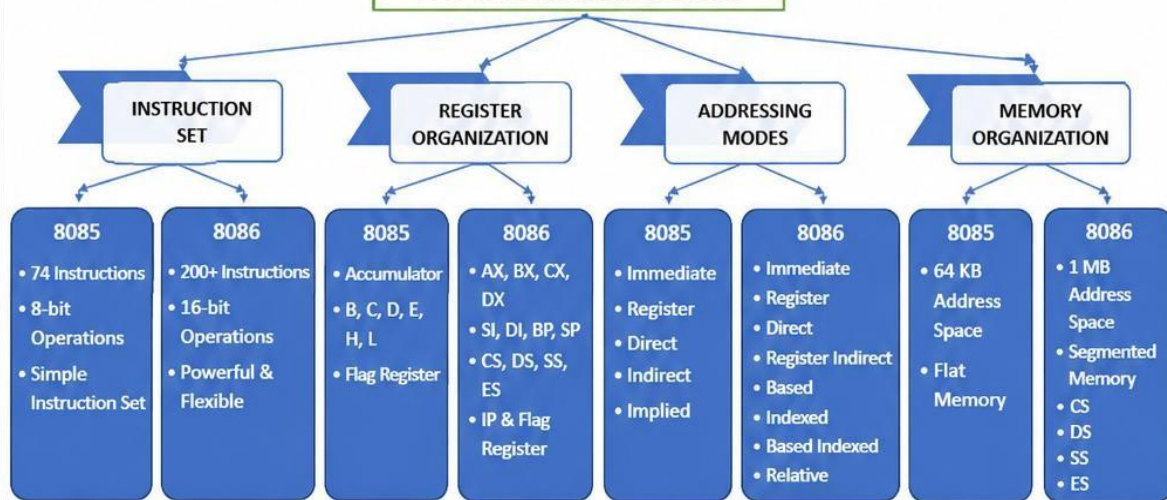
- Increase Bandwidth
- Reduce Contention
 - Allow parallel Transfers
- Better scalability & latency

POSSIBLE DELAYS / BOTTLENECKS

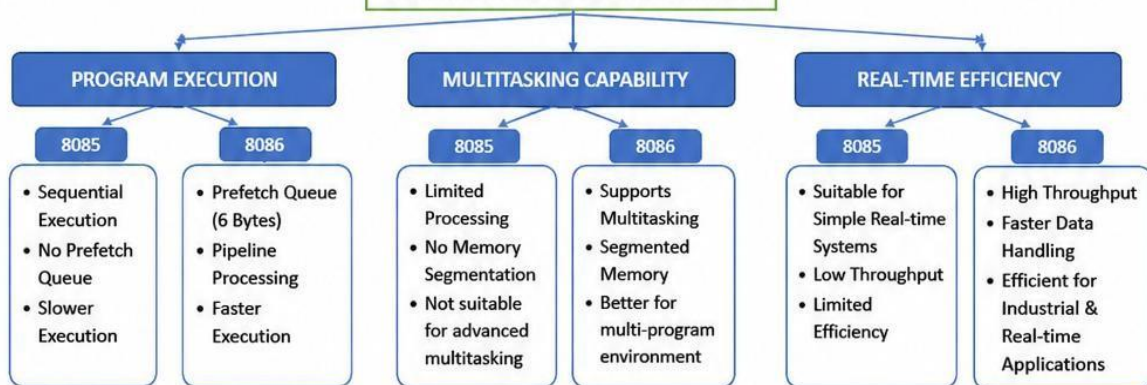
1. Bus Contention
2. Limited Bus Bandwidth
3. Slow Memory Access
4. I/O Device Latency



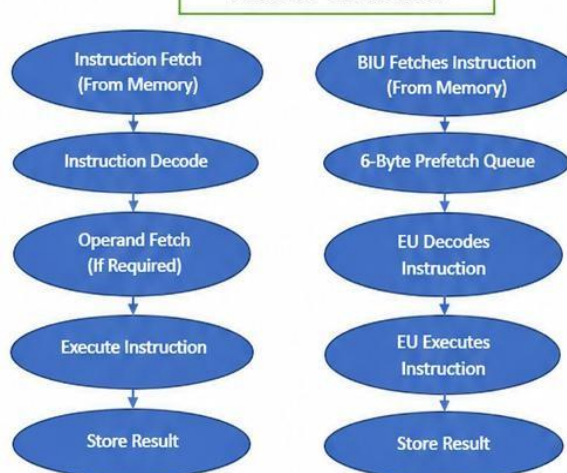
8085 vs 8086 ARCHITECTURES



ARCHITECTURAL FEATURES IMPACT



FLOW OF EXECUTION



PROGRAM EXECUTION FLOW (COMMON CONCEPT)

- Fetch Instruction
- Decode Instruction
- Execute Instruction
- Memory Access
- Store Result

HOW 8086 IMPROVES PERFORMANCE

- Larger Instruction Set
- More Registers – Less Memory Access
- 1 MB Memory with Segmentation
- 6-Byte Prefetch Queue
- Faster Execution
- Better Multitasking Support
- High Throughput
- Efficient for Real-time Applications

POSSIBLE LIMITATIONS / BOTTLENECKS

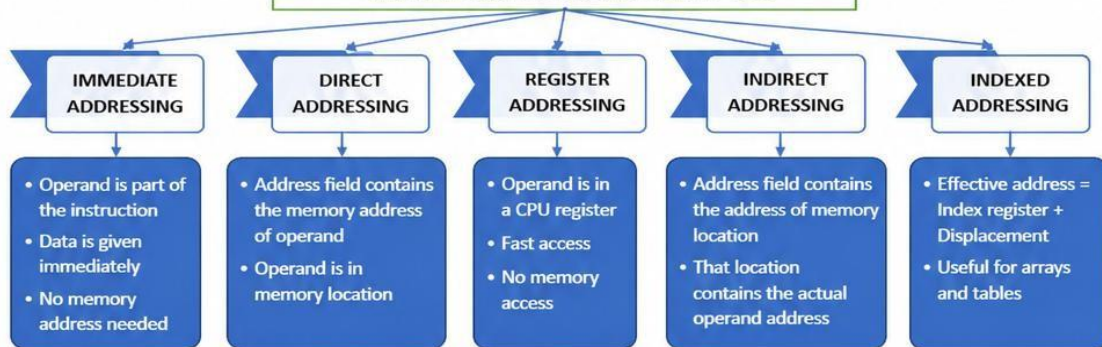
8085

- Limited to 64 KB Memory
- No Memory Segmentation
- Fewer Registers
- Slower Execution
- Not Suitable for Advanced Multitasking

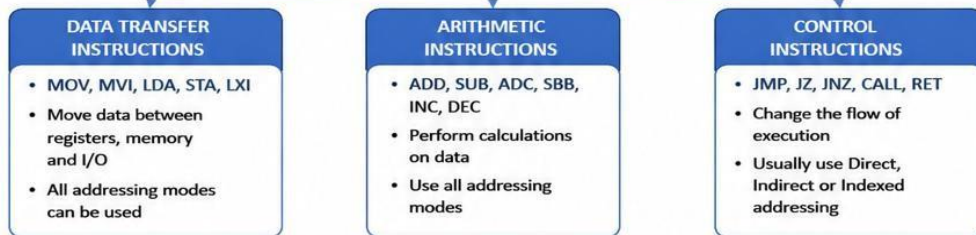
8086

- Segmentation Increases Complexity
- More Hardware & Cost
- Complex Programming & Debugging
- Segment Overlap May Cause Errors

ADDRESSING MODES AND INSTRUCTION TYPES THEIR IMPACT AND REAL-WORLD USE



INSTRUCTION TYPES



EFFECT OF ADDRESSING MODES

ADDRESSING MODE	MEMORY ACCESS TIME	INSTRUCTION SIZE	EXECUTION SPEED	REMARKS
IMMEDIATE	No memory access (Fastest)	Larger (contains data)	High	Best for constants and small data
DIRECT	One memory access (Slower than register)	Medium (address field)	Medium	Good for global variables
REGISTER	No memory access (Fastest)	Small (register number)	Very High (Fastest)	Best for temporary data
INDIRECT	Two memory accesses (Slowest)	Medium	Low (Slowest)	Used for pointers and dynamic data
INDEXED	One memory access + index calculation	Medium to Large (index + displacement)	Medium to High	Best for arrays, tables, buffers

REAL-WORLD SCENARIOS



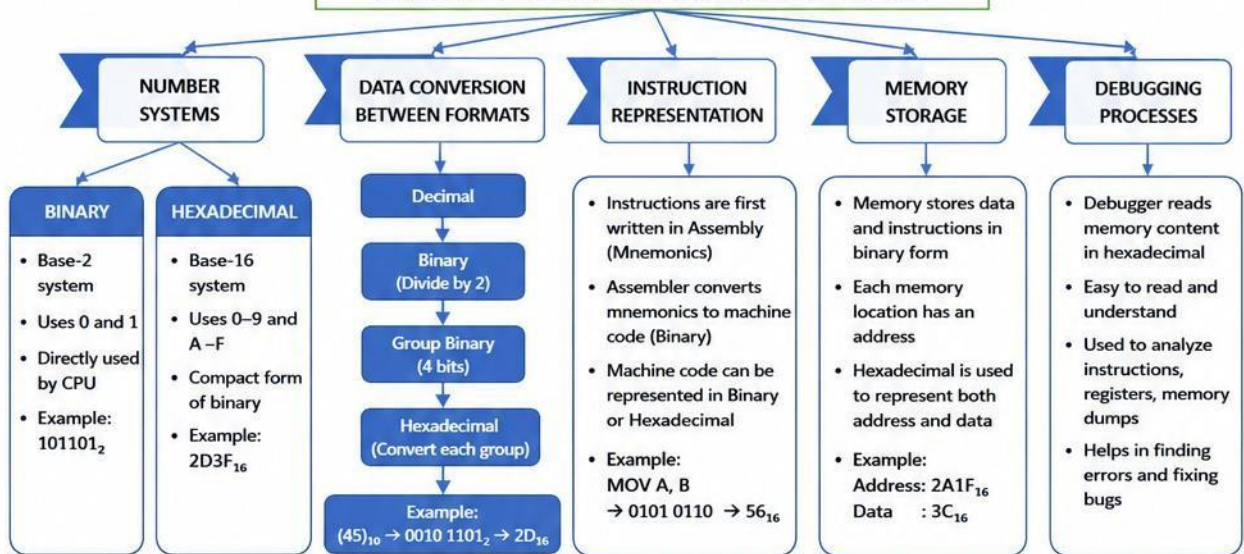
HOW IT AFFECTS PROGRAM PERFORMANCE



KEY TAKEAWAYS

- Register addressing is the fastest due to no memory access.
- Immediate addressing is best for constant values.
- Direct addressing is simple and widely used for global data.
- Indirect addressing adds flexibility but increases time.
- Indexed addressing is ideal for arrays and structured data.
- Proper addressing mode selection improves execution speed and memory efficiency.

NUMBER SYSTEMS, INSTRUCTION REPRESENTATION, MEMORY STORAGE AND DEBUGGING PROCESS



HOW CPU INTERPRETS AND EXECUTES INSTRUCTIONS

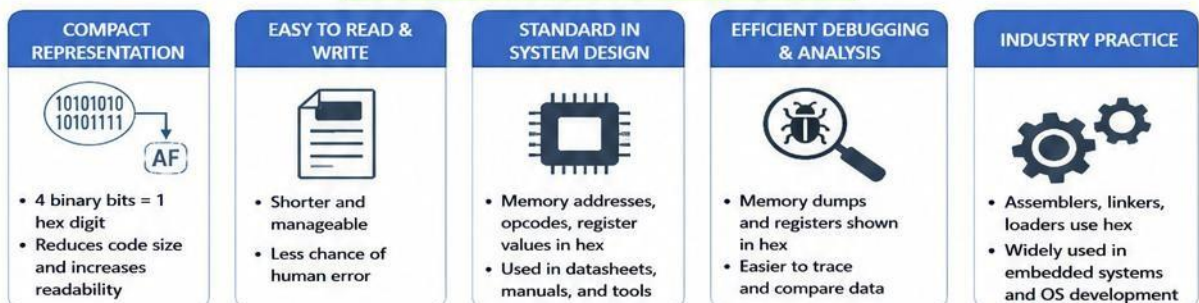


CPU works on Binary. Hexadecimal is used by humans for easy representation and understanding.

BINARY vs HEXADECIMAL REPRESENTATION

ASPECT	BINARY	HEXADECIMAL	EXAMPLE COMPARISON
Base	2	16	Binary: 1101 1010 Hex : DA
Digits Used	0, 1	0-9, A-F	Hex digits represent 4 bits each
Representation Size	Large (more bits)	Compact (fewer digits)	32-bit binary = 8 hex digits
Readability	Difficult for humans	Easy and shorthand	0x1A3F is easier than 0001 1010 0011 1111
Usage	Used by CPU	Used by Programmers, Debuggers	Memory addresses, Machine code, Registers

WHY HEXADECIMAL IS COMMONLY USED



DEBUGGING WORKFLOW USING HEXADECIMAL



KEY TAKEAWAYS

- Binary is the language of the CPU; hexadecimal is the language of humans.
- Instructions are stored in memory in binary but represented in hexadecimal for convenience.
- Hexadecimal simplifies memory addresses, data representation, and debugging.
- Understanding number systems and their conversion is essential for system design and low-level programming.